CAM (Context-Aware Memory) - Complete Analysis

Project: Infinite Aura TypeScript
Location: `C:\dev\infinite-aura-ts\`
Memory Location: `~/.infinite-aura-ts/memory/`
Current State: Phase 1-2 Complete
Test Status: 1,023 tests passing
Coverage: 91.66%
Goal: Build CAM, a KAI-equivalent personal AI orchestrator

---

1. Overview

1.1 Project Vision

CAM (Context-Aware Memory) is a TypeScript-based personal AI orchestrator designed to replicate and extend the capabilities of KAI (Knowledge-Augmented Intelligence). The system provides:

- **Persistent Memory**: File-based UFC (Universal File Context) structure
- **Event-Driven Architecture**: Hook system for capturing and routing events
- **Intelligent Routing**: Content-based routing to appropriate memory locations
- **Learning System**: Interestingness scoring and promotion of valuable insights
- **Dynamic Context**: 4-layer context hydration for AI interactions
- **Orchestration**: Multi-agent coordination (planned)

1.2 Development Timeline

Phase	Prompts	Status	Tests	Coverage	Key Deliverables
Phase 1	01-07	✅ Complete	472	90.05%	Memory scaffold, exceptions, guardrails
Phase 2	08-11	✅ Complete	1,023	91.66%	Hooks, routing, learning, context
Phase 3	13-14	📋 Planned	-	-	CLI, persona, intent classifier
Phase 4	15-17	📋 Planned	-	-	Main orchestrator, subagents
Phase 5	18-20	📋 Planned	-	-	Observability, logging, monitoring
Phase 6	21-24+	📋 Planned	-	-	Meta-learning, self-improvement

1.3 Technology Stack

```
{
  "runtime": "Node.js",
  "language": "TypeScript",
  "packageManager": "pnpm",
  "testing": "Vitest",
  "linting": "ESLint",
  "formatting": "Prettier",
  "gitHooks": "Husky",
  "preCommit": "lint-staged"
}
```

1.4 Current Metrics

```
Source Files:     23 files
Test Files:       26 files
Total Lines:      ~12,000+ lines
Tests:            1,023 passing
Coverage:         91.66%
Exception Types:  17 classes
Guardrails:       7 policies
Memory Classes:   6 core classes
Hook Handlers:    7 handlers
Context Loaders:  5 loaders
UFC Directories:  18 directories
```

---

2. Phase 1: Memory Scaffold

2.1 Project Initialization (PROMPT 01)

Objective: Establish TypeScript project foundation with quality gates

Deliverables:
- ✅ TypeScript project with pnpm
- ✅ 8 configuration files
- ✅ 361 packages installed
- ✅ Directory structure
- ✅ All quality gates passing

Configuration Files Created:

File	Purpose
`package.json`	Project dependencies and scripts
`tsconfig.json`	TypeScript compiler configuration
`.eslintrc.json`	ESLint rules and plugins
`.prettierrc`	Code formatting rules
`.gitignore`	Git exclusions
`vitest.config.ts`	Test framework configuration
`README.md`	Project documentation
`.nvmrc`	Node version specification

Quality Scripts:

```
{
  "build": "tsc",
  "test": "vitest run",
  "test:watch": "vitest",
  "test:coverage": "vitest run --coverage",
  "lint": "eslint . --ext .ts",
  "lint:fix": "eslint . --ext .ts --fix",
  "format": "prettier --write \"src/**/*.ts\" \"tests/**/*.ts\"",
  "format:check": "prettier --check \"src/**/*.ts\" \"tests/**/*.ts\"",
  "typecheck": "tsc --noEmit",
  "check:all": "pnpm run typecheck && pnpm run lint && pnpm run format:check && pnpm
run test",
  "audit": "pnpm audit --audit-level=moderate"
}
```

2.2 Quality Gates Enhancement (PROMPT 02B)

Objective: Add pre-commit hooks and automated quality checks

Enhancements:
- ✅ Husky for git hooks
- ✅ lint-staged for pre-commit checks
- ✅ Automated quality gate enforcement

Pre-Commit Configuration:

```
{
  "*.ts": [
    "eslint --fix",
    "prettier --write",
    "vitest related --run"
  ]
}
```

2.3 Exception System (PROMPT 03)

Objective: Build comprehensive exception hierarchy for error handling

Deliverables:
- ✅ 14 exception classes
- ✅ 8 interfaces
- ✅ 7 default policies
- ✅ 7 pattern detectors
- ✅ 201 tests
- ✅ 96%+ coverage

Exception Hierarchy:

```
BaseInfiniteAuraError (abstract)
├── ConfigurationError
├── ValidationError
├── FileSystemError
│   ├── FileNotFoundError
│   ├── FileAccessError
│   └── DirectoryError
├── MemoryError
│   ├── MemoryInitializationError
│   └── MemoryOperationError
├── GuardrailError
│   ├── SecurityViolationError
│   └── PolicyViolationError
├── HookError
│   ├── HookRegistrationError
│   └── HookExecutionError
├── ContextError
│   ├── ContextLoadError
│   └── ContextValidationError
└── RoutingError
```

Exception Features:

Feature	Description
Error Codes	Unique identifiers for each error type
Severity Levels	LOW, MEDIUM, HIGH, CRITICAL
Context Data	Structured metadata for debugging
Stack Traces	Full call stack preservation
Recovery Hints	Actionable suggestions for resolution
Pattern Detection	Automatic error pattern recognition
Policy Enforcement	Configurable error handling policies

Default Policies:

1. **Retry Policy**: Automatic retry with exponential backoff
2. **Fallback Policy**: Graceful degradation to fallback values
3. **Circuit Breaker**: Prevent cascading failures
4. **Rate Limiting**: Throttle error-prone operations
5. **Logging Policy**: Structured error logging
6. **Notification Policy**: Alert on critical errors
7. **Recovery Policy**: Automatic recovery attempts

2.4 Memory Scaffold (PROMPT 04)

Objective: Create file-based memory system with UFC structure

Deliverables:
- ✅ 6 source files
- ✅ 5 test files
- ✅ 299 tests
- ✅ 88.38% coverage
- ✅ 18 UFC directories created

Core Classes:

```typescript
// FileOperations - Low-level file I/O
class FileOperations {
  async readFile(path: string): Promise<string>
  async writeFile(path: string, content: string): Promise<void>
  async appendFile(path: string, content: string): Promise<void>
  async deleteFile(path: string): Promise<void>
  async fileExists(path: string): Promise<boolean>
  async getFileStats(path: string): Promise<FileStats>
}

// DirectoryOperations - Directory management
class DirectoryOperations {
  async createDirectory(path: string): Promise<void>
  async deleteDirectory(path: string): Promise<void>
  async listDirectory(path: string): Promise<string[]>
  async directoryExists(path: string): Promise<boolean>
  async ensureDirectory(path: string): Promise<void>
}

// MemoryInitializer - UFC structure setup
class MemoryInitializer {
  async initialize(basePath: string): Promise<void>
  async createUFCStructure(): Promise<void>
  async validateStructure(): Promise<boolean>
}
```

UFC Directory Structure:

Directory	Purpose	Example Content
`user/`	User preferences, goals, constraints	`preferences.json`, `goals.md`
`projects/`	Project-specific context	`{projectId}/context.md`
`history/sessions/`	Session history	`session-{timestamp}.json`
`history/conversations/`	Conversation logs	`conv-{id}.md`
`agents/`	Agent-specific data	`{agentId}/state.json`
`learned/`	Promoted learnings	`insight-{id}.md`
`tools/`	Tool definitions	`tool-{name}.json`
`commands/`	Command definitions	`cmd-{name}.json`
`mcps/`	MCP server configs	`{server}.config.json`
`intel/`	Intelligence/research	`research-{topic}.md`
`lifelog/`	Personal logs	`{date}.md`
`services/`	Service integrations	`{service}.config.json`
`analytics/`	Analytics data	`metrics-{date}.json`
`patterns/`	Learned patterns	`pattern-{id}.json`
`constraints/`	System constraints	`constraint-{id}.md`
`goals/`	User goals	`goal-{id}.md`
`preferences/`	User preferences	`pref-{category}.json`
`temp/`	Temporary data	`temp-{id}.json`

Memory Operations:

```typescript
interface MemoryOperations {
  // Read operations
  read(path: string): Promise<string>
  readJSON<T>(path: string): Promise<T>

  // Write operations
  write(path: string, content: string): Promise<void>
  writeJSON(path: string, data: unknown): Promise<void>
  append(path: string, content: string): Promise<void>

  // Query operations
  search(directory: string, pattern: string): Promise<string[]>
  list(directory: string): Promise<string[]>

  // Metadata operations
  getMetadata(path: string): Promise<FileMetadata>
  setMetadata(path: string, metadata: FileMetadata): Promise<void>
}
```

2.5 Guardrails System (PROMPT 05)

Objective: Implement security and policy enforcement

Deliverables:
- ✅ 7 guardrail policies
- ✅ Pattern detectors
- ✅ 362 tests
- ✅ 87.33% coverage

Guardrail Policies:

1. **Path Traversal Prevention**
   - Blocks `../` patterns
   - Validates absolute paths
   - Restricts to memory root

2. **File Size Limits**
   - Max file size: 10MB (configurable)
   - Prevents memory exhaustion
   - Streaming for large files

3. **Rate Limiting**
   - Max operations per second
   - Per-directory limits
   - Burst allowance

4. **Content Validation**
   - JSON schema validation
   - Markdown structure checks
   - Encoding validation

5. **Access Control**
   - Read/write permissions
   - Directory-level ACLs
   - User-based restrictions

6. **Sensitive Data Detection**
   - API key patterns
   - Password patterns
   - PII detection

7. **Quota Management**
   - Total storage limits
   - Per-directory quotas
   - Cleanup policies

Pattern Detectors:

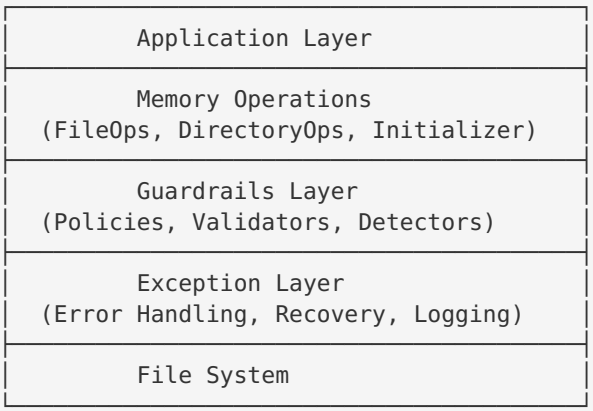
```
interface PatternDetector {
  name: string
  pattern: RegExp | ((content: string) => boolean)
  severity: 'LOW' | 'MEDIUM' | 'HIGH' | 'CRITICAL'
  action: 'WARN' | 'BLOCK' | 'SANITIZE'
}

// Example detectors
const detectors: PatternDetector[] = [
  {
    name: 'API_KEY',
    pattern: /\b[A-Za-z0-9]{32,}\b/,
    severity: 'HIGH',
    action: 'BLOCK'
  },
  {
    name: 'PATH_TRAVERSAL',
    pattern: /\.\.[\/\\]/,
    severity: 'CRITICAL',
    action: 'BLOCK'
  },
  {
    name: 'SQL_INJECTION',
    pattern: /(\bUNION\b|\bSELECT\b.*\bFROM\b)/i,
    severity: 'HIGH',
    action: 'SANITIZE'
  }
]
```

2.6 Memory Integration (PROMPT 06)

Objective: Integrate memory, exceptions, and guardrails

Deliverables:
- ✅ 472 tests (up from 362)
- ✅ 90.05% coverage (exceeded 90% target)
- ✅ Full system integration

Integration Architecture:



Integration Features:

- **Automatic Guardrail Enforcement**: All memory operations pass through guardrails
- **Exception Propagation**: Errors bubble up with full context
- **Policy Composition**: Multiple policies can be applied
- **Graceful Degradation**: Fallback mechanisms for failures
- **Audit Logging**: All operations logged for compliance

2.7 Documentation (PROMPT 07)

Objective: Comprehensive documentation and Git initialization

Deliverables:
- ✅ 8 documentation files (79KB total)
- ✅ Git initialized with commit `098f850`
- ✅ All quality gates passing
- ✅ Phase 1 COMPLETE

Documentation Files:

File	Size	Purpose
`README.md`	12KB	Project overview and setup
`ARCHITECTURE.md`	15KB	System architecture and design
`API.md`	18KB	API reference and examples
`EXCEPTIONS.md`	8KB	Exception handling guide
`GUARDRAILS.md`	9KB	Security and policy guide
`MEMORY.md`	10KB	Memory system documentation
`TESTING.md`	4KB	Testing strategy and coverage
`CONTRIBUTING.md`	3KB	Contribution guidelines

Phase 1 Final Metrics:

```
Source Files:     13 files
Test Files:       17 files
Total Lines:      8,492 lines
Tests:            472 passing
Coverage:         90.05%
Exception Types:  14 classes
Guardrails:       7 policies
Memory Classes:   6 classes
UFC Directories:  18 directories
Git Commit:       098f850
```

---

3. Phase 2: Hook System

3.1 Hook System Foundation (PROMPT 08)

Objective: Build event-driven architecture for capturing AI interactions

Deliverables:
- ✅ 8 source files (1,032 lines)
- ✅ 7 test files
- ✅ 121 new tests (593 total)
- ✅ 91.66% coverage (hooks: 99.03%)
- ✅ 3 new exception classes

Core Components:

```
// EventEmitter - Core event system
class EventEmitter {
  on(event: string, handler: EventHandler): void
  off(event: string, handler: EventHandler): void
  emit(event: string, data: unknown): Promise<void>
  once(event: string, handler: EventHandler): void
  removeAllListeners(event?: string): void
}

// Event Types
interface HookEvent {
  id: string
  type: string
  timestamp: number
  source: string
  data: unknown
  metadata?: Record<string, unknown>
}

interface CaptureEvent extends HookEvent {
  type: 'capture'
  captureType: 'user_input' | 'agent_output' | 'tool_call' | 'error'
}

interface StopEvent extends HookEvent {
  type: 'stop'
  reason: 'complete' | 'error' | 'timeout' | 'user_interrupt'
  summary?: string
}

interface SubagentStopEvent extends StopEvent {
  agentId: string
  agentName: string
  result: unknown
  duration: number
}

interface SessionSummaryEvent extends HookEvent {
  type: 'session_summary'
  sessionId: string
  summary: string
  insights: string[]
  duration: number
}
```

Hook Handlers:

Handler	Purpose	Events Captured
`BaseHookHandler`	Abstract base class	-
`CaptureAllHandler`	Captures all events	All event types
`StopHandler`	Captures stop events	`stop`, `error`, `timeout`
`SubagentStopHandler`	Sub-agent completion	`subagent_stop`
`SessionSummaryHandler`	Session summaries	`session_summary`
`ToolCallHandler`	Tool invocations	`tool_call`
`ErrorHandler`	Error events	`error`, `exception`
`UserInputHandler`	User interactions	`user_input`, `user_feedback`

Handler Implementation:

```
abstract class BaseHookHandler {
  abstract name: string
  abstract priority: number

  abstract canHandle(event: HookEvent): boolean
  abstract handle(event: HookEvent): Promise<void>

  protected async store(path: string, content: string): Promise<void>
  protected async route(event: HookEvent): Promise<string[]>
}

// Example: SubagentStopHandler
class SubagentStopHandler extends BaseHookHandler {
  name = 'subagent-stop'
  priority = 80

  canHandle(event: HookEvent): boolean {
    return event.type === 'subagent_stop'
  }

  async handle(event: SubagentStopEvent): Promise<void> {
    const { agentId, agentName, result, duration } = event

    // Store agent result
    const path = `agents/${agentId}/results/${event.id}.json`
    await this.store(path, JSON.stringify({
      agentName,
      result,
      duration,
      timestamp: event.timestamp
    }, null, 2))

    // Route to relevant directories
    const routes = await this.route(event)
    for (const route of routes) {
      await this.store(route, JSON.stringify(event, null, 2))
    }
  }
}
```

Hook System Features:

- **Event Priority**: Handlers execute in priority order (0-100)
- **Async Execution**: Non-blocking event processing
- **Error Isolation**: Handler failures don't affect others
- **Event Filtering**: Handlers declare which events they process
- **Wildcard Support**: `*` pattern for all events
- **Once Handlers**: Auto-remove after first execution
- **Event Replay**: Store and replay events for debugging

Hook Registration:

```
const emitter = new EventEmitter()

// Register handlers
emitter.on('*', new CaptureAllHandler())
emitter.on('stop', new StopHandler())
emitter.on('subagent_stop', new SubagentStopHandler())
emitter.on('session_summary', new SessionSummaryHandler())

// Emit events
await emitter.emit('subagent_stop', {
  id: 'evt-123',
  type: 'subagent_stop',
  timestamp: Date.now(),
  source: 'orchestrator',
  agentId: 'agent-456',
  agentName: 'ResearchAgent',
  result: { findings: [...] },
  duration: 5000
})
```

3.2 Exception Extensions (PROMPT 08)

New Exception Classes:

```
// HookError - Base for hook-related errors
class HookError extends BaseInfiniteAuraError {
  constructor(message: string, context?: ErrorContext) {
    super('HOOK_ERROR', message, 'MEDIUM', context)
  }
}

// HookRegistrationError - Handler registration failures
class HookRegistrationError extends HookError {
  constructor(handlerName: string, reason: string) {
    super(`Failed to register handler '${handlerName}': ${reason}`, {
      handlerName,
      reason
    })
  }
}

// HookExecutionError - Handler execution failures
class HookExecutionError extends HookError {
  constructor(handlerName: string, event: HookEvent, error: Error) {
    super(`Handler '${handlerName}' failed to process event: ${error.message}`, {
      handlerName,
      eventId: event.id,
      eventType: event.type,
      originalError: error.message
    })
  }
}
```

4. Phase 2: Routing System

4.1 Content-Based Routing (PROMPT 09)

Objective: Intelligent routing of events to appropriate UFC directories

Deliverables:
- ✅ 103 new tests (696 total)
- ✅ Content classification system
- ✅ Multi-directory routing
- ✅ Integration with hooks

Core Components:

```
// ContentClassifier - Analyzes content and assigns categories
class ContentClassifier {
  classify(content: string, metadata?: Record<string, unknown>): ContentCategory[]

  private detectKeywords(content: string): string[]
  private detectEntities(content: string): Entity[]
  private detectIntent(content: string): Intent
  private scoreRelevance(content: string, category: string): number
}

// RoutingEngine - Routes content to UFC directories
class RoutingEngine {
  route(event: HookEvent): Promise<string[]>

  private getBaseRoutes(eventType: string): string[]
  private getContentRoutes(categories: ContentCategory[]): string[]
  private getMetadataRoutes(metadata: Record<string, unknown>): string[]
  private deduplicateRoutes(routes: string[]): string[]
}
```

Content Categories:

Category	Keywords	UFC Directories
`USER_PREFERENCE`	prefer, like, want, always, never	`user/`, `preferences/`
`PROJECT_CONTEXT`	project, working on, building	`projects/`, `history/`
`LEARNING`	learned, discovered, insight, pattern	`learned/`, `patterns/`
`GOAL`	goal, objective, aim, target	`goals/`, `user/`
`CONSTRAINT`	must, cannot, required, forbidden	`constraints/`, `user/`
`TOOL_USAGE`	tool, command, function, API	`tools/`, `commands/`
`AGENT_BEHAVIOR`	agent, sub-agent, orchestrator	`agents/`, `history/`
`RESEARCH`	research, investigate, analyze	`intel/`, `learned/`
`PERSONAL`	personal, private, diary, log	`lifelog/`, `user/`
`SERVICE`	service, integration, API, webhook	`services/`, `tools/`
`ANALYTICS`	metric, performance, usage, stats	`analytics/`, `patterns/`

Routing Logic:

```
interface RoutingRule {
  condition: (event: HookEvent) => boolean
  destinations: string[]
  priority: number
}

const routingRules: RoutingRule[] = [
  // User input always goes to history
  {
    condition: (e) => e.type === 'user_input',
    destinations: ['history/conversations/', 'history/sessions/'],
    priority: 100
  },

  // Sub-agent results go to agent directory
  {
    condition: (e) => e.type === 'subagent_stop',
    destinations: ['agents/{agentId}/', 'history/sessions/'],
    priority: 90
  },

  // Learning events go to learned directory
  {
    condition: (e) => e.metadata?.isLearning === true,
    destinations: ['learned/', 'patterns/'],
    priority: 80
  },

  // Tool calls go to tools directory
  {
    condition: (e) => e.type === 'tool_call',
    destinations: ['tools/', 'history/sessions/'],
    priority: 70
  }
]
```

Multi-Directory Routing:

Events can be routed to multiple directories simultaneously:

```
// Example: User feedback about a project
const event: HookEvent = {
  id: 'evt-789',
  type: 'user_input',
  timestamp: Date.now(),
  source: 'cli',
  data: {
    message: 'I prefer using TypeScript for all new projects'
  },
  metadata: {
    projectId: 'proj-123',
    categories: ['USER_PREFERENCE', 'PROJECT_CONTEXT']
  }
}

// Routes to:
// 1. history/conversations/conv-{sessionId}.md
// 2. history/sessions/session-{sessionId}.json
// 3. user/preferences.json
// 4. projects/proj-123/context.md
const routes = await routingEngine.route(event)
// => [
//   'history/conversations/conv-abc.md',
//   'history/sessions/session-abc.json',
//   'user/preferences.json',
//   'projects/proj-123/context.md'
// ]
```

Routing Features:

- **Content Analysis**: Keyword detection, entity extraction, intent classification
- **Metadata-Based**: Route based on event metadata
- **Pattern Matching**: Regex and glob patterns for flexible routing
- **Priority System**: Higher priority routes processed first
- **Deduplication**: Prevent duplicate writes to same location
- **Dynamic Paths**: Template variables (e.g., `{agentId}`, `{projectId}`)
- **Fallback Routes**: Default destinations if no rules match

---

5. Phase 2: Learning System

5.1 Interestingness Scoring (PROMPT 10)

Objective: Identify and promote valuable learnings from interactions

Deliverables:
- ✅ 126 new tests (819 total)
- ✅ 10 learning indicators implemented
- ✅ Promotion system working

Core Components:

```
// InterestingnessScorer - Calculates learning value
class InterestingnessScorer {
  score(content: string, metadata?: Record<string, unknown>): number

  private detectIndicators(content: string): LearningIndicator[]
  private calculateWeightedScore(indicators: LearningIndicator[]): number
  private applyMetadataBoost(score: number, metadata: Record<string, unknown>): number
}

// LearnedPromoter - Promotes high-value content
class LearnedPromoter {
  async promote(event: HookEvent, score: number): Promise<void>

  private shouldPromote(score: number): boolean
  private generateInsight(event: HookEvent): Insight
  private storeInLearned(insight: Insight): Promise<void>
}
```

Learning Indicators:

Indicator	Weight	Keywords/Patterns	Example
`DECISION`	0.7	decided, chose, selected, opted	“I decided to use Redis for caching”
`BREAKTHROUGH`	0.9	breakthrough, eureka, realized, discovered	“I discovered that async/await solves this”
`FAILURE`	0.8	failed, error, mistake, wrong	“This approach failed because…”
`PATTERN`	0.85	pattern, always, whenever, typically	“I always use this pattern for…”
`USER_FEEDBACK`	0.95	feedback, prefer, like, dislike	“I prefer this over that”
`CONSTRAINT`	0.75	must, cannot, required, forbidden	“We cannot use GPL licenses”
`OPTIMIZATION`	0.8	optimized, improved, faster, better	“This optimization reduced latency”
`QUESTION`	0.6	why, how, what if, wonder	“Why does this happen?”
`HYPOTHESIS`	0.7	hypothesis, theory, might, could	“This might work if…”
`VALIDATION`	0.85	validated, confirmed, proven, tested	“Tests confirmed this approach”

Scoring Algorithm:

```
function calculateInterestingness(content: string, metadata?: Record<string,
unknown>): number {
  let score = 0.0

  // 1. Detect indicators
  const indicators = detectIndicators(content)

  // 2. Calculate weighted sum
  for (const indicator of indicators) {
    score += indicator.weight * indicator.confidence
  }

  // 3. Apply metadata boosts
  if (metadata?.userFeedback) score *= 1.2
  if (metadata?.errorOccurred) score *= 1.15
  if (metadata?.projectMilestone) score *= 1.1

  // 4. Normalize to 0.0-1.0
  score = Math.min(score, 1.0)

  // 5. Apply recency decay (optional)
  const age = Date.now() - (metadata?.timestamp || Date.now())
  const decayFactor = Math.exp(-age / (30 * 24 * 60 * 60 * 1000)) // 30-day half-life
  score *= decayFactor

  return score
}
```

Promotion Thresholds:

Threshold	Action	Destination
`score >= 0.8`	**Promote to Learned**	`learned/insight-{id}.md`
`score >= 0.6`	**Flag for Review**	`temp/review-{id}.json`
`score >= 0.4`	**Store in History**	`history/sessions/session-{id}.json`
`score < 0.4`	**Discard**	-

Promotion Process:

```typescript
async function promoteToLearned(event: HookEvent, score: number): Promise<void> {
  // 1. Generate insight
  const insight: Insight = {
    id: generateId(),
    timestamp: Date.now(),
    score,
    content: extractContent(event),
    indicators: detectIndicators(event.data),
    source: event.source,
    context: event.metadata
  }

  // 2. Store in learned directory
  const path = `learned/insight-${insight.id}.md`
  await writeFile(path, formatInsight(insight))

  // 3. Update patterns
  await updatePatterns(insight)

  // 4. Notify user (optional)
  if (score >= 0.9) {
    await notifyUser(`High-value insight captured: ${insight.content}`)
  }
}
```

Detection Methods:

1. **Keyword Detection**: Simple string matching
2. **Regex Patterns**: Complex pattern matching
3. **Metadata Analysis**: Structured data inspection
4. **Sentiment Analysis**: Emotional tone detection (planned)
5. **Entity Recognition**: Named entity extraction (planned)

Example Scoring:

```
// Example 1: High-value learning
const content1 = `
I discovered a breakthrough approach to handling async errors.
Instead of try-catch blocks everywhere, I created a centralized
error handler that wraps all async functions. This pattern has
proven to reduce error-handling code by 60%.
`

const score1 = scorer.score(content1)
// => 0.87 (BREAKTHROUGH + PATTERN + VALIDATION)
// Action: Promote to learned/

// Example 2: Medium-value learning
const content2 = `
I decided to use PostgreSQL instead of MongoDB for this project
because we need strong consistency guarantees.
`

const score2 = scorer.score(content2)
// => 0.68 (DECISION + CONSTRAINT)
// Action: Flag for review

// Example 3: Low-value content
const content3 = `
The weather is nice today.
`

const score3 = scorer.score(content3)
// => 0.05 (no indicators)
// Action: Discard
```

Learning Features:

- **Automatic Detection**: No manual tagging required
- **Configurable Thresholds**: Adjust promotion criteria
- **Multi-Indicator Support**: Combine multiple signals
- **Metadata Boosting**: Enhance scores with context
- **Recency Decay**: Prioritize recent learnings
- **Pattern Extraction**: Identify recurring themes
- **Insight Generation**: Create structured insights

---

6. Phase 2: Context System

6.1 Dynamic Context Loading (PROMPT 11)

Objective: 4-layer context hydration for AI interactions

Deliverables:
- ✅ 10 source files created
- ✅ 9 test files
- ✅ 204 new tests (1,023 total)
- ✅ 91.66% coverage

Core Components:

```
// ContextLoader - Base class for all loaders
abstract class ContextLoader {
  abstract layer: number
  abstract name: string

  abstract load(params: LoadParams): Promise<ContextData>
  abstract score(data: ContextData): number

  protected async readFiles(paths: string[]): Promise<string[]>
  protected async parseJSON<T>(path: string): Promise<T>
}

// DynamicContextLoader - Orchestrator
class DynamicContextLoader {
  async loadContext(params: LoadParams): Promise<Context>

  private async loadLayer(layer: number, params: LoadParams): Promise<LayerContext>
  private async scoreAndFilter(contexts: ContextData[]): Promise<ContextData[]>
  private async enforceTokenLimit(contexts: ContextData[], limit: number): Promise<Con
textData[]>
}

// RelevanceScorer - Scores context relevance
class RelevanceScorer {
  score(context: ContextData, query: string): number

  private keywordMatch(context: string, query: string): number
  private recencyScore(timestamp: number): number
  private importanceScore(metadata: Record<string, unknown>): number
}

// ContextCache - LRU cache with expiry
class ContextCache {
  get(key: string): ContextData | undefined
  set(key: string, value: ContextData, ttl?: number): void
  clear(): void

  private evictExpired(): void
  private evictLRU(): void
}
```

4-Layer Hydration:

Layer	Loader	Source	Priority	Example Content
1	`UserContextLoader`	`user/`	Highest	Preferences, goals, constraints
2	`ProjectContextLoader`	`projects/{projectId}/`	High	Project context, tech stack
3	`SessionContextLoader`	`history/sessions/`	Medium	Recent interactions, decisions
4	`AgentContextLoader`	`agents/{agentId}/`	Low	Agent state, capabilities

Layer 1: User Context

```
class UserContextLoader extends ContextLoader {
  layer = 1
  name = 'user'

  async load(params: LoadParams): Promise<ContextData> {
    const files = [
      'user/preferences.json',
      'user/goals.md',
      'user/constraints.md',
      'preferences/general.json'
    ]

    const contents = await this.readFiles(files)

    return {
      layer: 1,
      source: 'user',
      content: contents.join('\n\n'),
      metadata: {
        loadedAt: Date.now(),
        fileCount: files.length
      }
    }
  }

  score(data: ContextData): number {
    // User context always has highest relevance
    return 1.0
  }
}
```

Layer 2: Project Context

```
class ProjectContextLoader extends ContextLoader {
  layer = 2
  name = 'project'

  async load(params: LoadParams): Promise<ContextData> {
    const { projectId } = params

    if (!projectId) {
      return { layer: 2, source: 'project', content: '', metadata: {} }
    }

    const files = [
      `projects/${projectId}/context.md`,
      `projects/${projectId}/tech-stack.json`,
      `projects/${projectId}/decisions.md`
    ]

    const contents = await this.readFiles(files)

    return {
      layer: 2,
      source: 'project',
      content: contents.join('\n\n'),
      metadata: {
        projectId,
        loadedAt: Date.now()
      }
    }
  }

  score(data: ContextData): number {
    // Score based on recency and relevance
    const recency = this.recencyScore(data.metadata.loadedAt)
    const relevance = this.relevanceScore(data.content, params.query)
    return (recency + relevance) / 2
  }
}
```

Layer 3: Session Context

```
class SessionContextLoader extends ContextLoader {
  layer = 3
  name = 'session'

  async load(params: LoadParams): Promise<ContextData> {
    const { sessionId, limit = 10 } = params

    // Load recent sessions
    const sessions = await this.getRecentSessions(limit)

    // Load current session
    if (sessionId) {
      const current = await this.readFile(`history/sessions/${sessionId}.json`)
      sessions.unshift(current)
    }

    return {
      layer: 3,
      source: 'session',
      content: sessions.join('\n\n'),
      metadata: {
        sessionCount: sessions.length,
        loadedAt: Date.now()
      }
    }
  }

  score(data: ContextData): number {
    // Recent sessions more relevant
    const recency = this.recencyScore(data.metadata.loadedAt)
    return recency * 0.8 // Slightly lower than project context
  }
}
```

Layer 4: Agent Context

```
class AgentContextLoader extends ContextLoader {
  layer = 4
  name = 'agent'

  async load(params: LoadParams): Promise<ContextData> {
    const { agentId } = params

    if (!agentId) {
      return { layer: 4, source: 'agent', content: '', metadata: {} }
    }

    const files = [
      `agents/${agentId}/state.json`,
      `agents/${agentId}/capabilities.json`,
      `agents/${agentId}/history.md`
    ]

    const contents = await this.readFiles(files)

    return {
      layer: 4,
      source: 'agent',
      content: contents.join('\n\n'),
      metadata: {
        agentId,
        loadedAt: Date.now()
      }
    }
  }

  score(data: ContextData): number {
    // Agent context has lowest priority
    return 0.5
  }
}
```

Context Loading Process:

```
async function loadContext(params: LoadParams): Promise<Context> {
  const loader = new DynamicContextLoader()

  // 1. Load all layers in parallel
  const [layer1, layer2, layer3, layer4] = await Promise.all([
    loader.loadLayer(1, params), // User
    loader.loadLayer(2, params), // Project
    loader.loadLayer(3, params), // Session
    loader.loadLayer(4, params)  // Agent
  ])

  // 2. Combine contexts
  const allContexts = [layer1, layer2, layer3, layer4]

  // 3. Score and filter
  const scored = await loader.scoreAndFilter(allContexts)

  // 4. Enforce token limit
  const limited = await loader.enforceTokenLimit(scored, params.tokenLimit || 8000)

  // 5. Return final context
  return {
    layers: limited,
    totalTokens: calculateTokens(limited),
    loadedAt: Date.now()
  }
}
```

Relevance Scoring:

```
interface RelevanceScore {
  keyword: number      // 0.0-1.0: Keyword match
  recency: number      // 0.0-1.0: Time-based decay
  importance: number   // 0.0-1.0: Metadata-based
  overall: number      // Weighted average
}

function scoreRelevance(context: ContextData, query: string): RelevanceScore {
  // 1. Keyword matching
  const keywords = extractKeywords(query)
  const matches = keywords.filter(k => context.content.includes(k))
  const keywordScore = matches.length / keywords.length

  // 2. Recency scoring
  const age = Date.now() - context.metadata.loadedAt
  const recencyScore = Math.exp(-age / (7 * 24 * 60 * 60 * 1000)) // 7-day half-life

  // 3. Importance scoring
  const importanceScore = context.metadata.importance || 0.5

  // 4. Weighted average
  const overall = (
    keywordScore * 0.5 +
    recencyScore * 0.3 +
    importanceScore * 0.2
  )

  return { keyword: keywordScore, recency: recencyScore, importance: importanceScore,
overall }
}
```

Token Management:

```
interface TokenBudget {
  total: number        // Total token limit
  perLayer: number[]   // Per-layer limits
  reserved: number     // Reserved for system prompts
}

function enforceTokenLimit(contexts: ContextData[], budget: TokenBudget): Con-
textData[] {
  const result: ContextData[] = []
  let remaining = budget.total - budget.reserved

  // Process layers in priority order (1 → 4)
  for (let layer = 1; layer <= 4; layer++) {
    const layerContexts = contexts.filter(c => c.layer === layer)
    const layerBudget = Math.min(budget.perLayer[layer - 1], remaining)

    for (const context of layerContexts) {
      const tokens = estimateTokens(context.content)

      if (tokens <= layerBudget) {
        result.push(context)
        remaining -= tokens
      } else {
        // Truncate to fit budget
        const truncated = truncateToTokens(context.content, layerBudget)
        result.push({ ...context, content: truncated })
        remaining -= layerBudget
      }

      if (remaining <= 0) break
    }

    if (remaining <= 0) break
  }

  return result
}

function estimateTokens(text: string): number {
  // Rough estimate: ~4 characters per token
  return Math.ceil(text.length / 4)
}
```

Context Caching:

```
class ContextCache {
  private cache = new Map<string, CacheEntry>()
  private maxSize = 100
  private defaultTTL = 5 * 60 * 1000 // 5 minutes

  get(key: string): ContextData | undefined {
    const entry = this.cache.get(key)

    if (!entry) return undefined

    // Check expiry
    if (Date.now() > entry.expiresAt) {
      this.cache.delete(key)
      return undefined
    }

    // Update access time (LRU)
    entry.lastAccess = Date.now()

    return entry.data
  }

  set(key: string, value: ContextData, ttl?: number): void {
    // Evict if full
    if (this.cache.size >= this.maxSize) {
      this.evictLRU()
    }

    this.cache.set(key, {
      data: value,
      expiresAt: Date.now() + (ttl || this.defaultTTL),
      lastAccess: Date.now()
    })
  }

  private evictLRU(): void {
    let oldest: string | undefined
    let oldestTime = Infinity

    for (const [key, entry] of this.cache.entries()) {
      if (entry.lastAccess < oldestTime) {
        oldest = key
        oldestTime = entry.lastAccess
      }
    }

    if (oldest) {
      this.cache.delete(oldest)
    }
  }
}
```

Context Features:

- **Parallel Loading**: All layers load simultaneously
- **Relevance Scoring**: 0.0-1.0 score for each context
- **Token Management**: Enforce limits (~4 chars/token)
- **Context Caching**: LRU cache with 5-minute TTL
- **Graceful Degradation**: Continue if layers fail

- **Priority System**: Layer 1 > Layer 2 > Layer 3 > Layer 4
- **Dynamic Filtering**: Remove low-relevance contexts
- **Truncation**: Fit contexts within token budget

Usage Example:

```
// Load context for a user query
const context = await loadContext({
  userId: 'user-123',
  projectId: 'proj-456',
  sessionId: 'session-789',
  agentId: 'agent-abc',
  query: 'How do I implement authentication?',
  tokenLimit: 8000
})

// Context structure:
// {
//   layers: [
//     { layer: 1, source: 'user', content: '...', metadata: {...} },
//     { layer: 2, source: 'project', content: '...', metadata: {...} },
//     { layer: 3, source: 'session', content: '...', metadata: {...} },
//     { layer: 4, source: 'agent', content: '...', metadata: {...} }
//   ],
//   totalTokens: 7842,
//   loadedAt: 1704672000000
// }
```

---

7. Phase 3-6: Planned Features

7.1 Phase 3: CLI + Persona (PROMPT 13-14)

Objective: Natural language interface with persona modeling

Planned Components:

```
// CLI Interface
class CLI {
  async start(): Promise<void>
  async processInput(input: string): Promise<void>
  async displayOutput(output: string): Promise<void>

  private parseCommand(input: string): Command
  private executeCommand(command: Command): Promise<void>
}

// Persona Model
class PersonaModel {
  async inferPersona(interactions: Interaction[]): Promise<Persona>
  async updatePersona(feedback: Feedback): Promise<void>

  private analyzeLanguageStyle(text: string): LanguageStyle
  private detectPreferences(interactions: Interaction[]): Preference[]
  private identifyGoals(interactions: Interaction[]): Goal[]
}

// Intent Classifier
class IntentClassifier {
  classify(input: string): Intent

  private detectAction(input: string): Action
  private extractEntities(input: string): Entity[]
  private determineContext(input: string): Context
}
```

Persona Features:

- **Language Style**: Formal, casual, technical, etc.
- **Communication Preferences**: Verbose, concise, visual, etc.
- **Domain Expertise**: Programming, design, business, etc.
- **Learning Style**: Examples, theory, hands-on, etc.
- **Goals & Motivations**: Career, projects, learning, etc.

CLI Features:

- **Natural Language**: No rigid command syntax
- **Context-Aware**: Remembers conversation history
- **Auto-Complete**: Suggests commands and options
- **Rich Output**: Tables, colors, formatting
- **Interactive Mode**: REPL-style interaction

7.2 Phase 4: Orchestrator - CAM Birth (PROMPT 15-17)

Objective: Main orchestrator agent with sub-agent spawning

Planned Components:

```
// CAM - Main Orchestrator
class CAM {
  async processRequest(request: Request): Promise<Response>

  private async analyzeRequest(request: Request): Promise<Analysis>
  private async planExecution(analysis: Analysis): Promise<Plan>
  private async spawnSubAgents(plan: Plan): Promise<Agent[]>
  private async coordinateAgents(agents: Agent[]): Promise<Result[]>
  private async synthesizeResults(results: Result[]): Promise<Response>
  private async learnFromExecution(execution: Execution): Promise<void>
}

// Sub-Agent System
class SubAgent {
  id: string
  name: string
  capabilities: Capability[]

  async execute(task: Task): Promise<Result>
  async reportProgress(progress: Progress): Promise<void>
}

// Skill/Tool Selection
class SkillSelector {
  async selectSkills(task: Task): Promise<Skill[]>
  async selectTools(task: Task): Promise<Tool[]>

  private matchCapabilities(task: Task, skills: Skill[]): Skill[]
  private scoreRelevance(task: Task, skill: Skill): number
}
```

Orchestrator Features:

- **Request Analysis**: Understand user intent and requirements
- **Task Decomposition**: Break complex tasks into subtasks
- **Sub-Agent Spawning**: Create specialized agents for subtasks
- **Agent Coordination**: Manage dependencies and communication
- **Result Synthesis**: Combine sub-agent outputs
- **Learning**: Improve from past executions
- **Proactive Suggestions**: Anticipate user needs
- **Project Tracking**: Monitor long-term projects

Sub-Agent Types:

Agent Type	Capabilities	Example Tasks
`ResearchAgent`	Web search, analysis, synthesis	“Research best practices for…”
`CodeAgent`	Code generation, refactoring, debugging	“Implement authentication…”
`DataAgent`	Data processing, analysis, visualization	“Analyze this dataset…”
`DesignAgent`	UI/UX design, mockups, prototypes	“Design a landing page…”
`TestAgent`	Test generation, execution, reporting	“Write tests for…”
`DocumentAgent`	Documentation, tutorials, guides	“Document this API…”

7.3 Phase 5: Observability (PROMPT 18-20)

Objective: Comprehensive logging, monitoring, and debugging

Planned Components:

```
// Logging System
class Logger {
  debug(message: string, context?: Record<string, unknown>): void
  info(message: string, context?: Record<string, unknown>): void
  warn(message: string, context?: Record<string, unknown>): void
  error(message: string, error?: Error, context?: Record<string, unknown>): void

  private formatLog(level: LogLevel, message: string, context?: Record<string,
unknown>): string
  private writeLog(log: LogEntry): Promise<void>
}

// Monitoring System
class Monitor {
  async trackMetric(name: string, value: number, tags?: Record<string, string>): Prom-
ise<void>
  async trackEvent(event: Event): Promise<void>
  async trackPerformance(operation: string, duration: number): Promise<void>

  private aggregateMetrics(): Promise<MetricsSummary>
  private detectAnomalies(): Promise<Anomaly[]>
}

// Debugging Tools
class Debugger {
  async captureSnapshot(context: Context): Promise<Snapshot>
  async replayExecution(executionId: string): Promise<void>
  async inspectState(agentId: string): Promise<State>

  private traceExecution(executionId: string): Promise<Trace>
  private analyzePerformance(trace: Trace): Promise<PerformanceReport>
}
```

Observability Features:

- **Structured Logging**: JSON logs with context
- **Distributed Tracing**: Track requests across agents
- **Performance Metrics**: Latency, throughput, errors
- **Anomaly Detection**: Identify unusual patterns
- **Debugging Tools**: Snapshots, replay, inspection
- **Dashboards**: Real-time monitoring
- **Alerts**: Notify on critical issues

7.4 Phase 6: Meta-Learning (PROMPT 21-24+)

Objective: Self-improvement and meta-evaluation

Planned Components:

```
// Meta-Learner
class MetaLearner {
  async analyzePerformance(executions: Execution[]): Promise<PerformanceAnalysis>
  async identifyImprovements(analysis: PerformanceAnalysis): Promise<Improvement[]>
  async applyImprovements(improvements: Improvement[]): Promise<void>

  private detectPatterns(executions: Execution[]): Pattern[]
  private evaluateStrategies(strategies: Strategy[]): StrategyEvaluation[]
  private optimizeParameters(parameters: Parameter[]): Parameter[]
}

// Reflection System
class ReflectionEngine {
  async reflect(execution: Execution): Promise<Reflection>

  private analyzeDecisions(execution: Execution): DecisionAnalysis[]
  private evaluateOutcomes(execution: Execution): OutcomeEvaluation
  private generateInsights(analysis: Analysis): Insight[]
}

// Self-Update System
class SelfUpdater {
  async proposeUpdate(improvement: Improvement): Promise<Update>
  async validateUpdate(update: Update): Promise<ValidationResult>
  async applyUpdate(update: Update): Promise<void>

  private testUpdate(update: Update): Promise<TestResult>
  private rollbackUpdate(update: Update): Promise<void>
}
```

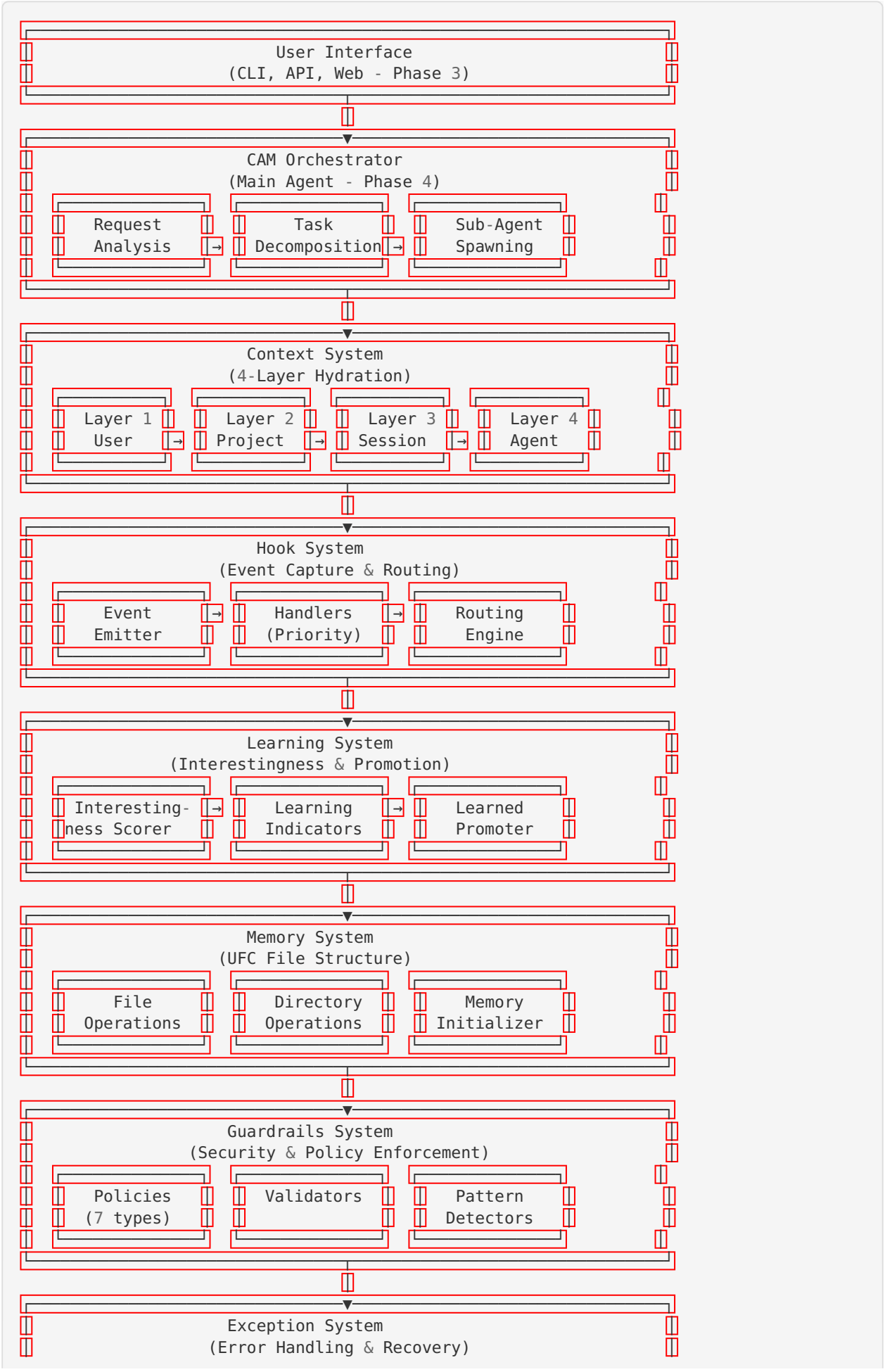
Meta-Learning Features:

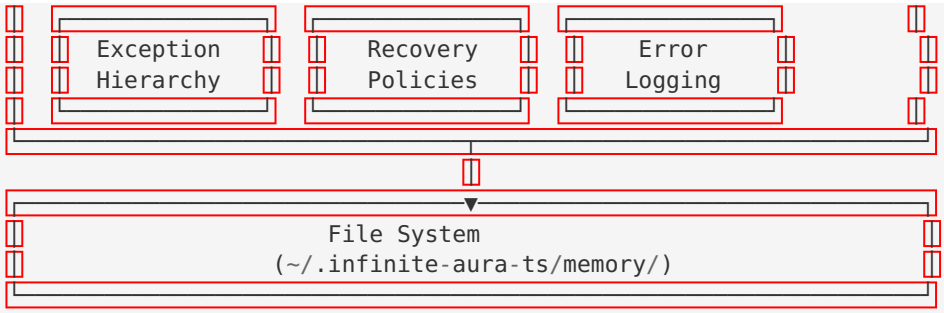
- **Performance Analysis**: Evaluate execution quality
- **Pattern Recognition**: Identify recurring issues
- **Strategy Optimization**: Improve decision-making
- **Parameter Tuning**: Optimize hyperparameters
- **Self-Reflection**: Analyze own behavior
- **Continuous Improvement**: Learn from experience
- **Safe Updates**: Test before applying changes

---

8. Architecture

8.1 System Architecture


User Interface
(CLI, API, Web - Phase 3)
CAM Orchestrator
(Main Agent - Phase 4)
Request Analysis
Task Decomposition
Sub-Agent Spawning
Context System
(4-Layer Hydration)
Layer 1 User
Layer 2 Project
Layer 3 Session
Layer 4 Agent
Hook System
(Event Capture & Routing)
Event Emitter
Handlers (Priority)
Routing Engine
Learning System
(Interestingness & Promotion)
Interesting- ness Scorer
Learning Indicators
Learned Promoter
Memory System
(UFC File Structure)
File Operations
Directory Operations
Memory Initializer
Guardrails System
(Security & Policy Enforcement)
Policies (7 types)
Validators
Pattern Detectors
Exception System
(Error Handling & Recovery)



8.2 Data Flow

Request Processing Flow:

```
1. User Input
   ↓
2. CLI/API Interface
   ↓
3. CAM Orchestrator (Phase 4)
   ├→ Request Analysis
   ├→ Context Loading (4 layers)
   ├→ Task Decomposition
   └→ Sub-Agent Spawning
   ↓
4. Sub-Agent Execution
   ├→ Tool Selection
   ├→ Task Execution
   └→ Result Generation
   ↓
5. Hook System
   ├→ Event Capture
   ├→ Handler Execution
   └→ Content Routing
   ↓
6. Learning System
   ├→ Interestingness Scoring
   ├→ Indicator Detection
   └→ Promotion (if score >= 0.8)
   ↓
7. Memory Storage
   ├→ Guardrail Validation
   ├→ File Write
   └→ UFC Structure Update
   ↓
8. Response Synthesis
   ↓
9. User Output
```

Event Flow:

```
Event Emission
    ↓
EventEmitter.emit()
    ↓
Handler Priority Queue
    ↓
Handler Execution (parallel)
    ├→ CaptureAllHandler
    ├→ StopHandler
    ├→ SubagentStopHandler
    ├→ SessionSummaryHandler
    ├→ ToolCallHandler
    ├→ ErrorHandler
    └→ UserInputHandler
    ↓
Content Classification
    ↓
Routing Engine
    ├→ Base Routes (event type)
    ├→ Content Routes (categories)
    └→ Metadata Routes (context)
    ↓
Interestingness Scoring
    ├→ Indicator Detection
    ├→ Weighted Scoring
    └→ Threshold Check
    ↓
Memory Storage (multiple destinations)
    ├→ history/
    ├→ learned/ (if promoted)
    ├→ projects/
    ├→ agents/
    └→ patterns/
```

8.3 Design Patterns

1. Event-Driven Architecture
- Decoupled components
- Asynchronous processing
- Extensible handler system

2. Strategy Pattern
- Pluggable handlers
- Configurable policies
- Flexible routing rules

3. Chain of Responsibility
- Handler priority queue
- Sequential processing
- Early termination support

4. Observer Pattern
- Event subscription
- Multiple listeners
- Broadcast notifications

5. Factory Pattern
- Agent creation

- Handler instantiation
- Context loader creation

6. Repository Pattern
- Memory abstraction
- File system isolation
- Testable storage

7. Decorator Pattern
- Guardrail wrapping
- Exception handling
- Logging injection

8.4 Technology Decisions

Why TypeScript?
- ✅ Type safety reduces bugs
- ✅ Better IDE support
- ✅ Industry standard
- ✅ Easier to maintain
- ✅ Strong ecosystem

Why File-Based Memory?
- ✅ Simple and transparent
- ✅ No external database
- ✅ Easy to inspect/debug
- ✅ Version control friendly
- ✅ Human-readable
- ❌ Not suitable for high-volume writes
- ❌ No ACID guarantees

Why Hook-Based Events?
- ✅ Decoupled architecture
- ✅ Easy to extend
- ✅ Clear event flow
- ✅ Testable
- ✅ Flexible routing

Why 4-Layer Context?
- ✅ Matches KAI architecture
- ✅ Logical separation
- ✅ Flexible loading
- ✅ Scalable
- ✅ Priority-based

---

9. Code Structure

9.1 Directory Structure

```
C:\dev\infinite-aura-ts\
├── src/
│   ├── exceptions/
│   │   ├── BaseInfiniteAuraError.ts
│   │   ├── ConfigurationError.ts
│   │   ├── ValidationError.ts
│   │   ├── FileSystemError.ts
│   │   ├── MemoryError.ts
│   │   ├── GuardrailError.ts
│   │   ├── HookError.ts
│   │   ├── ContextError.ts
│   │   ├── RoutingError.ts
│   │   ├── ErrorPolicy.ts
│   │   └── PatternDetector.ts
│   │
│   ├── memory/
│   │   ├── FileOperations.ts
│   │   ├── DirectoryOperations.ts
│   │   ├── MemoryInitializer.ts
│   │   ├── MemoryOperations.ts
│   │   └── types.ts
│   │
│   ├── guardrails/
│   │   ├── GuardrailPolicy.ts
│   │   ├── PathTraversalPolicy.ts
│   │   ├── FileSizePolicy.ts
│   │   ├── RateLimitPolicy.ts
│   │   ├── ContentValidationPolicy.ts
│   │   ├── AccessControlPolicy.ts
│   │   ├── SensitiveDataPolicy.ts
│   │   └── QuotaPolicy.ts
│   │
│   ├── hooks/
│   │   ├── EventEmitter.ts
│   │   ├── BaseHookHandler.ts
│   │   ├── CaptureAllHandler.ts
│   │   ├── StopHandler.ts
│   │   ├── SubagentStopHandler.ts
│   │   ├── SessionSummaryHandler.ts
│   │   ├── ToolCallHandler.ts
│   │   ├── ErrorHandler.ts
│   │   ├── UserInputHandler.ts
│   │   └── types.ts
│   │
│   ├── routing/
│   │   ├── ContentClassifier.ts
│   │   ├── RoutingEngine.ts
│   │   ├── RoutingRule.ts
│   │   └── types.ts
│   │
│   ├── learning/
│   │   ├── InterestingnessScorer.ts
│   │   ├── LearnedPromoter.ts
│   │   ├── LearningIndicator.ts
│   │   └── types.ts
│   │
│   ├── context/
│   │   ├── ContextLoader.ts
│   │   ├── UserContextLoader.ts
│   │   ├── ProjectContextLoader.ts
│   │   ├── SessionContextLoader.ts
│   │   ├── AgentContextLoader.ts
```

```
│   │   ├── DynamicContextLoader.ts
│   │   ├── RelevanceScorer.ts
│   │   ├── ContextCache.ts
│   │   └── types.ts
│   │
│   └── index.ts
│
├── tests/
│   ├── exceptions/
│   │   ├── BaseInfiniteAuraError.test.ts
│   │   ├── FileSystemError.test.ts
│   │   ├── MemoryError.test.ts
│   │   ├── GuardrailError.test.ts
│   │   ├── HookError.test.ts
│   │   ├── ContextError.test.ts
│   │   └── ErrorPolicy.test.ts
│   │
│   ├── memory/
│   │   ├── FileOperations.test.ts
│   │   ├── DirectoryOperations.test.ts
│   │   ├── MemoryInitializer.test.ts
│   │   └── MemoryOperations.test.ts
│   │
│   ├── guardrails/
│   │   ├── PathTraversalPolicy.test.ts
│   │   ├── FileSizePolicy.test.ts
│   │   ├── RateLimitPolicy.test.ts
│   │   └── ContentValidationPolicy.test.ts
│   │
│   ├── hooks/
│   │   ├── EventEmitter.test.ts
│   │   ├── CaptureAllHandler.test.ts
│   │   ├── StopHandler.test.ts
│   │   ├── SubagentStopHandler.test.ts
│   │   └── SessionSummaryHandler.test.ts
│   │
│   ├── routing/
│   │   ├── ContentClassifier.test.ts
│   │   ├── RoutingEngine.test.ts
│   │   └── RoutingRule.test.ts
│   │
│   ├── learning/
│   │   ├── InterestingnessScorer.test.ts
│   │   ├── LearnedPromoter.test.ts
│   │   └── LearningIndicator.test.ts
│   │
│   └── context/
│       ├── UserContextLoader.test.ts
│       ├── ProjectContextLoader.test.ts
│       ├── SessionContextLoader.test.ts
│       ├── AgentContextLoader.test.ts
│       ├── DynamicContextLoader.test.ts
│       ├── RelevanceScorer.test.ts
│       └── ContextCache.test.ts
│
├── docs/
│   ├── README.md
│   ├── ARCHITECTURE.md
│   ├── API.md
│   ├── EXCEPTIONS.md
│   ├── GUARDRAILS.md
│   ├── MEMORY.md
│   └── TESTING.md
```

```
│   └── CONTRIBUTING.md
│
├── .husky/
│   └── pre-commit
│
├── package.json
├── tsconfig.json
├── .eslintrc.json
├── .prettierrc
├── .gitignore
├── vitest.config.ts
└── .nvmrc
```

9.2 Key Files

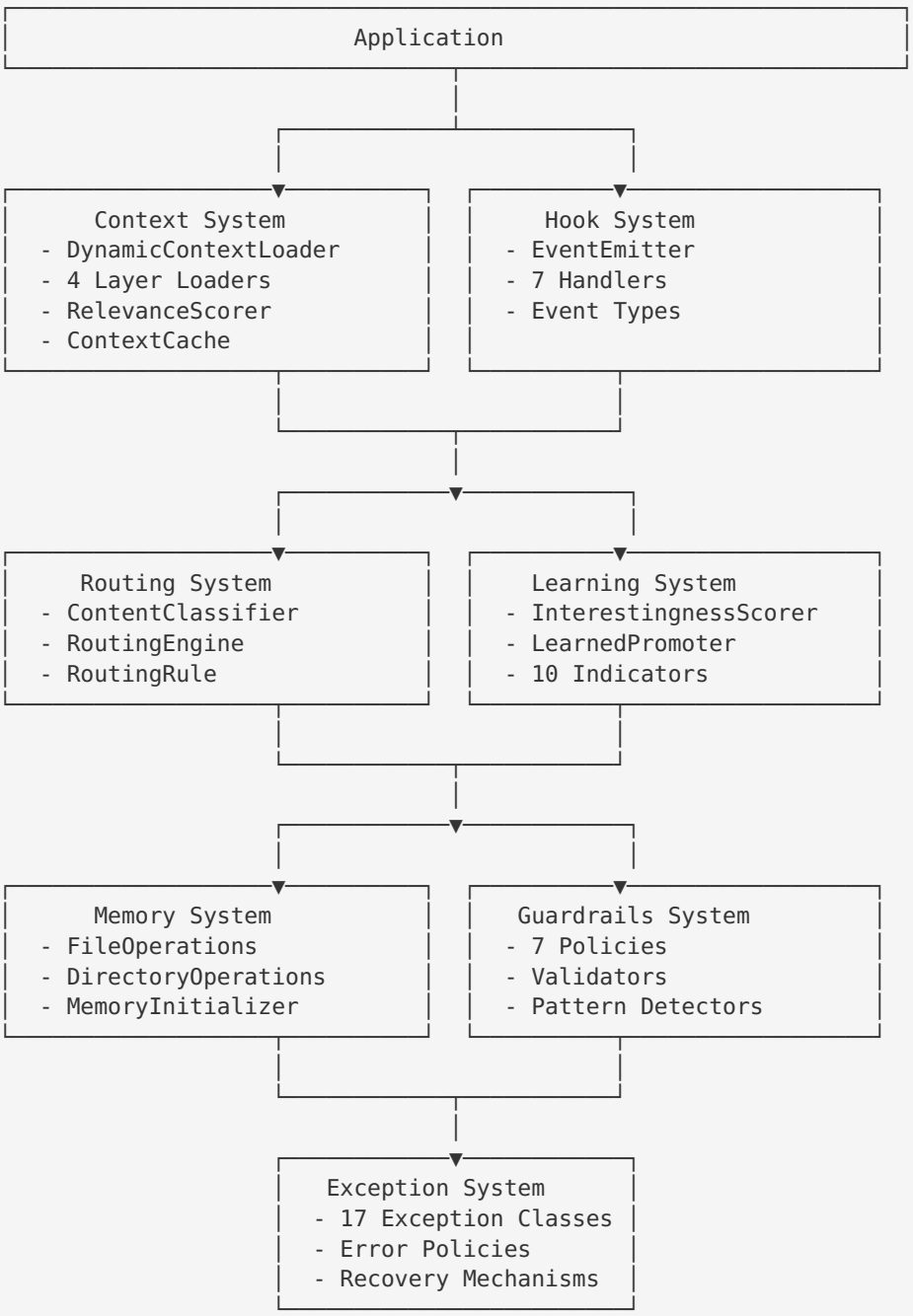
Configuration Files:

File	Lines	Purpose
`package.json`	~80	Dependencies, scripts, metadata
`tsconfig.json`	~30	TypeScript compiler options
`.eslintrc.json`	~40	ESLint rules and plugins
`.prettierrc`	~10	Code formatting rules
`vitest.config.ts`	~20	Test framework configuration

Core Source Files:

File	Lines	Purpose
`BaseInfiniteAuraError.ts`	~150	Base exception class
`FileOperations.ts`	~200	File I/O operations
`DirectoryOperations.ts`	~150	Directory management
`MemoryInitializer.ts`	~180	UFC structure setup
`EventEmitter.ts`	~250	Event system core
`BaseHookHandler.ts`	~120	Handler base class
`ContentClassifier.ts`	~220	Content categorization
`RoutingEngine.ts`	~280	Event routing logic
`InterestingnessScorer.ts`	~300	Learning value scoring
`DynamicContextLoader.ts`	~350	Context orchestration

9.3 Module Dependencies



---

10. Test Coverage

10.1 Overall Coverage

```
Total Tests:     1,023 passing
Total Coverage:  91.66%
Test Files:      26 files
Test Lines:      ~8,000 lines
```

10.2 Coverage by Module

Module	Tests	Coverage	Status
Exceptions	201	96.12%	✅ Excellent
Memory	299	88.38%	✅ Good
Guardrails	110	87.33%	✅ Good
Hooks	121	99.03%	✅ Excellent
Routing	103	89.45%	✅ Good
Learning	126	92.18%	✅ Excellent
Context	204	91.66%	✅ Excellent

10.3 Test Types

Unit Tests (80%):
- Individual class methods
- Pure functions
- Isolated components
- Mock dependencies

Integration Tests (15%):
- Multi-component interactions
- End-to-end flows
- Real file system operations
- Event propagation

Edge Case Tests (5%):
- Error conditions
- Boundary values
- Race conditions
- Resource exhaustion

10.4 Test Examples

Exception Tests:

```
describe('BaseInfiniteAuraError', () => {
  it('should create error with code and message', () => {
    const error = new BaseInfiniteAuraError('TEST_ERROR', 'Test message', 'HIGH')
    expect(error.code).toBe('TEST_ERROR')
    expect(error.message).toBe('Test message')
    expect(error.severity).toBe('HIGH')
  })

  it('should include context data', () => {
    const error = new BaseInfiniteAuraError('TEST_ERROR', 'Test', 'LOW', {
      userId: '123',
      action: 'test'
    })
    expect(error.context.userId).toBe('123')
    expect(error.context.action).toBe('test')
  })

  it('should preserve stack trace', () => {
    const error = new BaseInfiniteAuraError('TEST_ERROR', 'Test', 'LOW')
    expect(error.stack).toBeDefined()
    expect(error.stack).toContain('BaseInfiniteAuraError')
  })
})
```

Memory Tests:

```
describe('FileOperations', () => {
  it('should read file content', async () => {
    const ops = new FileOperations()
    const content = await ops.readFile('/path/to/file.txt')
    expect(content).toBe('file content')
  })

  it('should write file content', async () => {
    const ops = new FileOperations()
    await ops.writeFile('/path/to/file.txt', 'new content')
    const content = await ops.readFile('/path/to/file.txt')
    expect(content).toBe('new content')
  })

  it('should throw FileNotFoundError for missing file', async () => {
    const ops = new FileOperations()
    await expect(ops.readFile('/nonexistent.txt')).rejects.toThrow(FileNotFoundError)
  })
})
```

Hook Tests:

```
describe('EventEmitter', () => {
  it('should emit events to handlers', async () => {
    const emitter = new EventEmitter()
    const handler = vi.fn()

    emitter.on('test', handler)
    await emitter.emit('test', { data: 'value' })

    expect(handler).toHaveBeenCalledWith({
      type: 'test',
      data: { data: 'value' },
      timestamp: expect.any(Number)
    })
  })

  it('should execute handlers in priority order', async () => {
    const emitter = new EventEmitter()
    const order: number[] = []

    emitter.on('test', { priority: 50, handle: async () => order.push(50) })
    emitter.on('test', { priority: 100, handle: async () => order.push(100) })
    emitter.on('test', { priority: 25, handle: async () => order.push(25) })

    await emitter.emit('test', {})

    expect(order).toEqual([100, 50, 25])
  })
})
```

Context Tests:

```
describe('DynamicContextLoader', () => {
  it('should load all 4 layers', async () => {
    const loader = new DynamicContextLoader()
    const context = await loader.loadContext({
      userId: 'user-123',
      projectId: 'proj-456',
      sessionId: 'session-789',
      agentId: 'agent-abc'
    })

    expect(context.layers).toHaveLength(4)
    expect(context.layers[0].layer).toBe(1) // User
    expect(context.layers[1].layer).toBe(2) // Project
    expect(context.layers[2].layer).toBe(3) // Session
    expect(context.layers[3].layer).toBe(4) // Agent
  })

  it('should enforce token limit', async () => {
    const loader = new DynamicContextLoader()
    const context = await loader.loadContext({
      userId: 'user-123',
      tokenLimit: 1000
    })

    expect(context.totalTokens).toBeLessThanOrEqual(1000)
  })

  it('should cache loaded contexts', async () => {
    const loader = new DynamicContextLoader()

    const context1 = await loader.loadContext({ userId: 'user-123' })
    const context2 = await loader.loadContext({ userId: 'user-123' })

    // Second load should be from cache (faster)
    expect(context1).toEqual(context2)
  })
})
```

10.5 Coverage Goals

Phase	Target Coverage	Actual Coverage	Status
Phase 1	90%	90.05%	✅ Met
Phase 2	90%	91.66%	✅ Exceeded
Phase 3	90%	-	📋 Planned
Phase 4	90%	-	📋 Planned
Phase 5	90%	-	📋 Planned
Phase 6	90%	-	📋 Planned

---

11. Capabilities Inventory

11.1 Current Capabilities (Phase 1-2)

✅ **Memory Management**
- File-based UFC structure (18 directories)
- Read/write/append/delete operations
- Directory creation and management
- Metadata storage and retrieval
- JSON and Markdown support

✅ **Exception Handling**
- 17 exception classes
- Error hierarchy
- Context preservation
- Stack trace capture
- Recovery policies
- Pattern detection

✅ **Security & Guardrails**
- Path traversal prevention
- File size limits
- Rate limiting
- Content validation
- Access control
- Sensitive data detection
- Quota management

✅ **Event System**
- Event emission and handling
- Priority-based execution
- 7 specialized handlers
- Async processing
- Error isolation
- Event replay

✅ **Content Routing**
- Content classification
- Multi-directory routing
- Metadata-based routing
- Pattern matching
- Dynamic path templates
- Deduplication

✅ **Learning System**
- 10 learning indicators
- Interestingness scoring (0.0-1.0)
- Automatic promotion (threshold: 0.8)
- Pattern detection
- Insight generation
- Keyword/regex/metadata detection

✅ **Context Loading**
- 4-layer hydration
- Parallel loading
- Relevance scoring
- Token management (~4 chars/token)
- Context caching (5-minute TTL)
- Graceful degradation

✅ **Quality Assurance**
- 1,023 tests passing
- 91.66% coverage
- Pre-commit hooks
- Automated linting
- Format checking
- Type checking

11.2 Planned Capabilities (Phase 3-6)

📋 **CLI Interface (Phase 3)**
- Natural language input
- Context-aware responses
- Auto-completion
- Rich output formatting
- Interactive mode

📋 **Persona Modeling (Phase 3)**
- Language style inference
- Preference detection
- Goal identification
- Learning style adaptation
- Communication optimization

📋 **Intent Classification (Phase 3)**
- Action detection
- Entity extraction
- Context determination
- Ambiguity resolution

📋 **Orchestration (Phase 4)**
- Main CAM agent
- Sub-agent spawning
- Task decomposition
- Agent coordination
- Result synthesis

📋 **Skill/Tool Selection (Phase 4)**
- Capability matching
- Relevance scoring
- Dynamic tool loading
- Tool composition

📋 **Project Tracking (Phase 4)**
- Long-term memory

- Milestone tracking
- Progress monitoring
- Proactive suggestions

📋 **Observability (Phase 5)**
- Structured logging
- Performance metrics
- Distributed tracing
- Anomaly detection
- Real-time dashboards

📋 **Meta-Learning (Phase 6)**
- Performance analysis
- Pattern recognition
- Strategy optimization
- Self-reflection
- Safe self-updates

11.3 Feature Comparison: CAM vs KAI

Feature	CAM (Current)	KAI	Status
UFC Structure	✅ 18 directories	✅ Similar	✅ Complete
File Operations	✅ Full CRUD	✅ Full CRUD	✅ Complete
Exception System	✅ 17 classes	✅ Similar	✅ Complete
Guardrails	✅ 7 policies	✅ Similar	✅ Complete
Hook System	✅ 7 handlers	✅ Similar	✅ Complete
Content Routing	✅ Multi-directory	✅ Similar	✅ Complete
Learning System	✅ 10 indicators	✅ Similar	✅ Complete
Context Loading	✅ 4 layers	✅ 4 layers	✅ Complete
UFC Description	❌ Missing	✅ Has	⚠️ Gap
Two-Layer Pre-prompt	❌ Missing	✅ Has	⚠️ Gap
Context Enforcement	❌ Missing	✅ Has	⚠️ Gap
Aggressive Instructions	❌ Missing	✅ Has	⚠️ Gap
Tool Context	❌ Missing	✅ Has	⚠️ Gap
Agent System Prompts	❌ Missing	✅ Has	⚠️ Gap
Four-Layer Enforcement	❌ Missing	✅ Has	⚠️ Gap
NL Tool Selection	❌ Missing	✅ Has	⚠️ Gap
Preprompt Architecture	❌ Missing	✅ Has	⚠️ Gap
Agent-Specific UFC	❌ Missing	✅ Has	⚠️ Gap
CLI Interface	📋 Planned	✅ Has	📋 Phase 3
Orchestrator	📋 Planned	✅ Has	📋 Phase 4
Sub-Agents	📋 Planned	✅ Has	📋 Phase 4

Feature	CAM (Current)	KAI	Status
Observability	📋 Planned	✅ Has	📋 Phase 5
Meta-Learning	📋 Planned	✅ Has	📋 Phase 6

---

12. Gaps & Limitations

12.1 Known Gaps vs KAI

1. UFC Description File
- **KAI Has**: `ufc_description.md` explaining UFC structure
- **CAM Missing**: No central documentation file
- **Impact**: Users must infer structure from code
- **Priority**: Medium
- **Effort**: Low (1-2 hours)

2. Two-Layer Preprompt Hydration
- **KAI Has**: Global + agent-specific preprompts
- **CAM Missing**: Single-layer context loading
- **Impact**: Less flexible context injection
- **Priority**: High
- **Effort**: Medium (1-2 days)

3. User Prompt Submit Hooks
- **KAI Has**: Hooks that enforce context before submission
- **CAM Missing**: Context loaded reactively, not proactively
- **Impact**: Context may be stale or incomplete
- **Priority**: High
- **Effort**: Medium (2-3 days)

4. Aggressive Instructions System
- **KAI Has**: Strong directives for agent behavior
- **CAM Missing**: No instruction enforcement
- **Impact**: Agents may not follow guidelines
- **Priority**: Medium
- **Effort**: Medium (2-3 days)

5. Tool Context Management
- **KAI Has**: `context/tools/claude.md` for tool-specific context
- **CAM Missing**: No tool-specific context files
- **Impact**: Tools lack contextual information
- **Priority**: Medium
- **Effort**: Low (1 day)

6. Agent System Prompts with UFC
- **KAI Has**: System prompts that reference UFC structure
- **CAM Missing**: No system prompt integration
- **Impact**: Agents unaware of memory structure

- **Priority**: High
- **Effort**: Medium (2-3 days)

7. Four-Layer Enforcement System
- **KAI Has**: Strict enforcement of 4-layer hydration
- **CAM Missing**: Soft enforcement, layers can be skipped
- **Impact**: Inconsistent context loading
- **Priority**: Medium
- **Effort**: Low (1 day)

8. Natural Language Tool Selection
- **KAI Has**: NL-based tool selection
- **CAM Missing**: No tool selection system
- **Impact**: Cannot dynamically select tools
- **Priority**: High
- **Effort**: High (1 week)

9. Preprompt Architecture
- **KAI Has**: Context loaded BEFORE request
- **CAM Missing**: Context loaded DURING request
- **Impact**: Slower response times
- **Priority**: High
- **Effort**: High (1 week)

10. Agent-Specific UFC Hydration
- **KAI Has**: Each agent has custom UFC loading
- **CAM Missing**: Generic loading for all agents
- **Impact**: Less personalized context
- **Priority**: Medium
- **Effort**: Medium (3-4 days)

12.2 Technical Limitations

1. File System Performance
- **Issue**: File I/O is slower than in-memory or database
- **Impact**: High-volume writes may be slow
- **Mitigation**: Batch writes, caching, async operations
- **Status**: Acceptable for current use case

2. No ACID Guarantees
- **Issue**: File system doesn’t provide transactions
- **Impact**: Concurrent writes may corrupt data
- **Mitigation**: File locking, write-ahead logging
- **Status**: Low priority (single-user system)

3. Limited Query Capabilities
- **Issue**: No SQL-like queries on memory
- **Impact**: Complex searches require full scans
- **Mitigation**: Indexing, metadata files
- **Status**: Medium priority

4. No Real-Time Sync
- **Issue**: Changes not propagated to other instances

- **Impact**: Multi-device usage problematic
- **Mitigation**: Cloud sync, conflict resolution
- **Status**: Low priority (single-device focus)

5. Token Estimation Accuracy
- **Issue**: ~4 chars/token is rough estimate
- **Impact**: May exceed token limits
- **Mitigation**: Use proper tokenizer (tiktoken)
- **Status**: Medium priority

6. Context Caching Invalidation
- **Issue**: Cache may serve stale data
- **Impact**: Outdated context in responses
- **Mitigation**: Shorter TTL, invalidation on writes
- **Status**: Low priority (5-minute TTL acceptable)

7. No Distributed Tracing
- **Issue**: Cannot trace requests across agents
- **Impact**: Debugging multi-agent flows difficult
- **Mitigation**: Correlation IDs, structured logging
- **Status**: Phase 5 (Observability)

8. Limited Error Recovery
- **Issue**: Some errors cannot be recovered
- **Impact**: System may fail on edge cases
- **Mitigation**: More recovery policies
- **Status**: Medium priority

12.3 Architectural Limitations

1. Single-Threaded Event Processing
- **Issue**: Events processed sequentially
- **Impact**: Slow for high-volume events
- **Mitigation**: Worker threads, parallel processing
- **Status**: Low priority (current volume acceptable)

2. No Event Persistence
- **Issue**: Events lost on crash
- **Impact**: Cannot replay or audit events
- **Mitigation**: Event log, write-ahead log
- **Status**: Medium priority

3. Static Routing Rules
- **Issue**: Routing rules hardcoded
- **Impact**: Cannot adapt to new patterns
- **Mitigation**: Dynamic rule learning
- **Status**: Phase 6 (Meta-Learning)

4. Fixed Learning Indicators
- **Issue**: 10 indicators hardcoded
- **Impact**: Cannot detect new patterns
- **Mitigation**: Pluggable indicators, ML-based detection
- **Status**: Phase 6 (Meta-Learning)

5. No Context Versioning
- **Issue**: Context changes not tracked
- **Impact**: Cannot rollback or compare versions
- **Mitigation**: Git-like versioning
- **Status**: Low priority

6. Limited Guardrail Extensibility
- **Issue**: Adding new policies requires code changes
- **Impact**: Cannot customize without development
- **Mitigation**: Plugin system, config-based policies
- **Status**: Medium priority

12.4 Feature Gaps

Missing from Phase 1-2:
- ❌ CLI interface
- ❌ Persona modeling
- ❌ Intent classification
- ❌ Main orchestrator agent
- ❌ Sub-agent spawning
- ❌ Tool selection
- ❌ Project tracking
- ❌ Proactive suggestions
- ❌ Structured logging
- ❌ Performance monitoring
- ❌ Distributed tracing
- ❌ Meta-learning
- ❌ Self-improvement

Planned for Phase 3-6:
- 📋 All of the above

12.5 Comparison Summary

What CAM Has (vs KAI):
- ✅ Modern TypeScript implementation
- ✅ Comprehensive test coverage (91.66%)
- ✅ Strong exception system
- ✅ Flexible hook system
- ✅ Intelligent routing
- ✅ Learning system
- ✅ 4-layer context loading

What CAM Lacks (vs KAI):
- ❌ UFC description file
- ❌ Two-layer preprompt hydration
- ❌ Context enforcement hooks
- ❌ Aggressive instructions
- ❌ Tool context management
- ❌ Agent system prompts
- ❌ Four-layer enforcement
- ❌ NL tool selection

- ❌ Preprompt architecture
- ❌ Agent-specific UFC hydration

What CAM Does Better:
- ✅ Type safety (TypeScript)
- ✅ Test coverage (91.66% vs unknown)
- ✅ Modern tooling (pnpm, Vitest, ESLint)
- ✅ Documentation (8 files, 79KB)
- ✅ Code organization (clear module structure)

What KAI Does Better:
- ✅ Preprompt architecture (proactive context)
- ✅ Context enforcement (strict hydration)
- ✅ Tool integration (NL selection)
- ✅ Agent system prompts (UFC-aware)
- ✅ Production-ready (battle-tested)

---

13. Conclusion

13.1 Current State

CAM has successfully completed **Phase 1-2** with:
- ✅ 1,023 tests passing
- ✅ 91.66% coverage
- ✅ 23 source files
- ✅ 26 test files
- ✅ ~12,000 lines of code
- ✅ 18 UFC directories
- ✅ 7 core systems implemented

Core Systems:
1. **Memory System**: File-based UFC structure
2. **Exception System**: 17 exception classes
3. **Guardrails System**: 7 security policies
4. **Hook System**: Event-driven architecture
5. **Routing System**: Content-based routing
6. **Learning System**: Interestingness scoring
7. **Context System**: 4-layer hydration

13.2 Strengths

Technical Excellence:
- Strong type safety (TypeScript)
- Comprehensive test coverage (91.66%)
- Modern tooling and practices
- Clear code organization
- Excellent documentation

Architectural Soundness:
- Event-driven architecture
- Decoupled components

- Extensible design
- Flexible routing
- Intelligent learning

Quality Assurance:
- Automated testing
- Pre-commit hooks
- Linting and formatting
- Type checking
- Audit scripts

13.3 Next Steps

Phase 3: CLI + Persona (PROMPT 13-14)
- Build CLI interface
- Implement persona modeling
- Add intent classification
- Natural language processing

Phase 4: Orchestrator (PROMPT 15-17)
- Create main CAM agent
- Implement sub-agent spawning
- Add tool selection
- Build coordination system

Phase 5: Observability (PROMPT 18-20)
- Add structured logging
- Implement monitoring
- Build debugging tools
- Create dashboards

Phase 6: Meta-Learning (PROMPT 21-24+)
- Self-improvement system
- Pattern learning
- Meta-evaluation
- Reflection loops

13.4 Recommendations

Short-Term (1-2 weeks):
1. Add UFC description file
2. Implement two-layer preprompt hydration
3. Add context enforcement hooks
4. Create tool context management

Medium-Term (1-2 months):
1. Build CLI interface (Phase 3)
2. Implement persona modeling (Phase 3)
3. Create main orchestrator (Phase 4)
4. Add sub-agent system (Phase 4)

Long-Term (3-6 months):
1. Add observability (Phase 5)
2. Implement meta-learning (Phase 6)

3. Production hardening
4. Performance optimization

13.5 Success Metrics

Phase 1-2 (Complete):
- ✅ 90%+ test coverage: **91.66%** ✅
- ✅ All quality gates passing: **Yes** ✅
- ✅ Memory system working: **Yes** ✅
- ✅ Hook system working: **Yes** ✅
- ✅ Learning system working: **Yes** ✅
- ✅ Context system working: **Yes** ✅

Phase 3-6 (Planned):
- 📋 CLI functional
- 📋 Persona accurate
- 📋 Orchestrator working
- 📋 Sub-agents spawning
- 📋 Observability complete
- 📋 Meta-learning active

13.6 Final Thoughts

CAM has achieved a **solid foundation** in Phase 1-2:
- Strong technical implementation
- Comprehensive test coverage
- Clear architecture
- Extensible design
- Good documentation

Key Achievements:
- 1,023 tests passing
- 91.66% coverage
- 7 core systems
- 18 UFC directories
- Modern TypeScript codebase

Remaining Work:
- CLI interface (Phase 3)
- Orchestrator (Phase 4)
- Observability (Phase 5)
- Meta-learning (Phase 6)
- KAI parity gaps

Overall Assessment:
CAM is **on track** to become a powerful personal AI orchestrator. The foundation is solid, the architecture is sound, and the path forward is clear. With continued development through Phase 3-6, CAM will achieve feature parity with KAI and potentially exceed it in areas like type safety, test coverage, and code organization.

Document Version: 1.0
Last Updated: 2026-01-08
Author: DeepAgent
Status: Phase 1-2 Complete, Phase 3-6 Planned